

第2章 数字逻辑设计基础

用卡诺图化简下列函数：

$$F(A, B, C) = \prod M(0, 1, 6, 7)$$

解：

Step 1: $\bar{F}$ 的最大项表示

$$\bar{F}(A, B, C) = \prod M(2, 3, 4, 5)$$

Step 2: $\bar{F}$ 的最小项表示

$$\bar{F}(A, B, C) = \sum m(0, 1, 6, 7)$$

Step 3: 画出 $\bar{F}$ 的卡诺图并化简

AB \ C	00	01	11	10
0	1		1	
1	1		1	

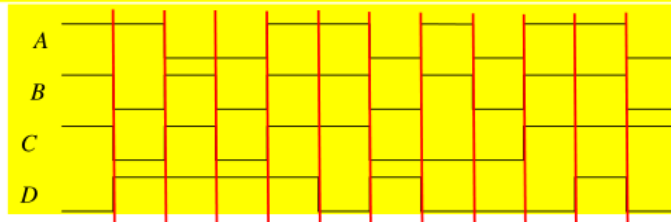
Step 4: 写出 $\bar{F}$ 的最简与或式

$$\bar{F}(A, B, C) = \bar{A}\bar{B} + AB$$

Step 5: 写出 F 的最简或与式

$$F(A, B, C) = (A + B)(\bar{A} + \bar{B})$$

已知某组合逻辑电路有 3 个输入和 1 个输出，其工作波形如下图所示。



a) 确定输入量和输出量，并简要给出理由。

b) 写出输入输出的逻辑关系式，化简为最简与或式，试用与门，或门，非门完成逻辑电路设计。

解：

a)

由 ABCD 为 1001 和 0001 可知 A 为输入

由 ABCD 为 1110 和 1100 可知 C 为输入

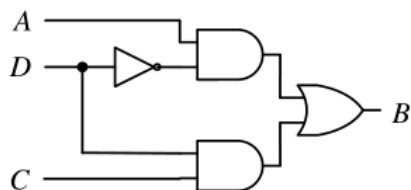
由 ABCD 为 0001 和 0000 可知 D 为输入

所以 A、C、D 为输入，B 为输出

b)

		B			
D	AC	00	01	11	10
	0			1	1
1	1		1	1	

$$B = CD + A\bar{D}$$



第 3 章 组合逻辑电路

试设计 4-bit 质数检测器。该电路有 4 个输入：N3、N2、N1 和 N0，对应于一个 4-bit 数字（N3 是最高位）和一个输出 P，当输入为质数时，P 输出 1，否则 P 输出 0。写出真值表及逻辑函数最小项表达式，用与、或、非逻辑门实现逻辑电路，写出 HDL 代码。

解：

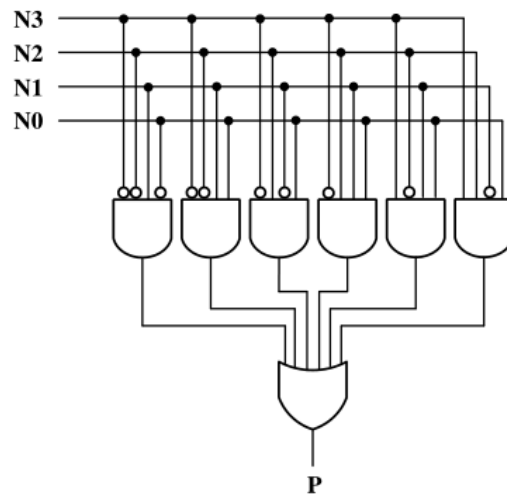
真值表

N3	N2	N1	N0	P	N3	N2	N1	N0	P
0	0	0	0	0	1	0	0	0	0
0	0	0	1	0	1	0	0	1	0
0	0	1	0	1	1	0	1	0	0
0	0	1	1	1	1	0	1	1	1
0	1	0	0	0	1	1	0	0	0
0	1	0	1	1	1	1	0	1	1
0	1	1	0	0	1	1	1	0	0
0	1	1	1	1	1	1	1	1	0

逻辑函数表达式

$$P = \bar{N}_3\bar{N}_2N_1\bar{N}_0 + \bar{N}_3\bar{N}_2N_1N_0 + \bar{N}_3N_2\bar{N}_1N_0 + \bar{N}_3N_2N_1N_0 + N_3\bar{N}_2N_1\bar{N}_0 + N_3N_2\bar{N}_1N_0$$

逻辑电路



HDL 代码

```
module prime(N3, N2, N1, N0, P);
    input N3, N2, N1, N0;
    output reg P;

    always @(*)
        case ({N3, N2, N1, N0})
            0, 1, 4, 6, 8, 9, 10, 12, 14, 15: P <= 1'b0;
            2, 3, 5, 7, 11, 13: P <= 1'b1;
        endcase
endmodule
```

已知真值表如下，A、B、C 为输入，F 为输出。使用以 A 为选择信号的 2 选 1 数据选择器和逻辑门电路来实现逻辑功能。

A	B	C	F
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

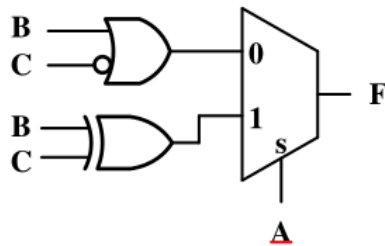
解：

<u>A</u>	<u>B</u>	<u>C</u>	F
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

} $A=0, F = B + \overline{C}$

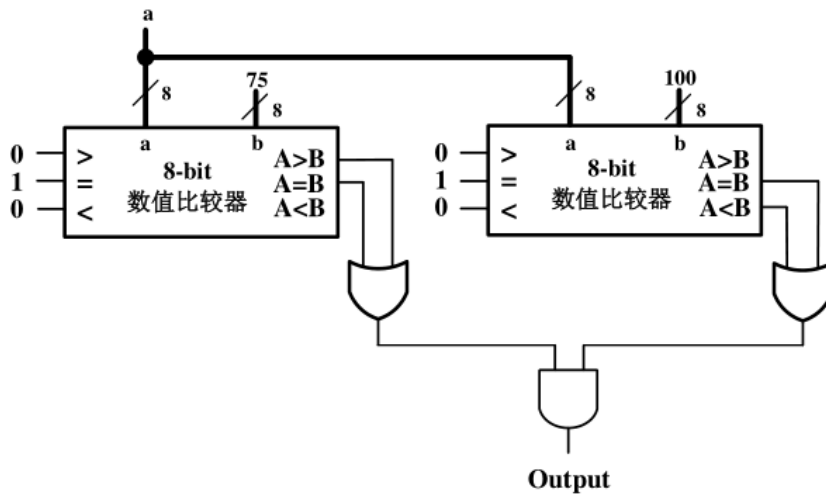
} $A=1, F = B \oplus C$

逻辑电路



用 8-bit 数值比较器和逻辑门设计一个电路，当 8-bit 输入 a 介于 75 和 100 之间（含 75 和 100）时，输出 1。

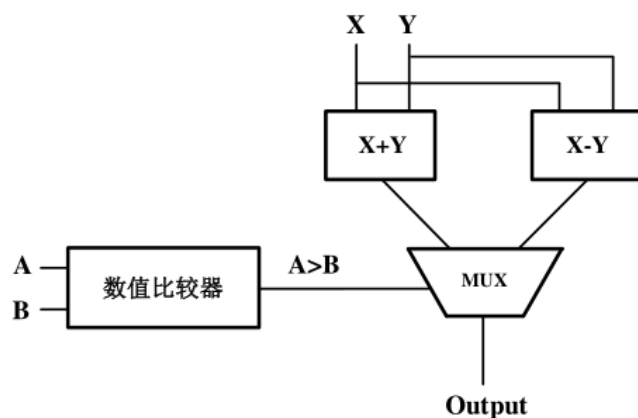
解：



选取比较器、多路选择器、加法器和减法器模块实现下列运算，并画出框图。

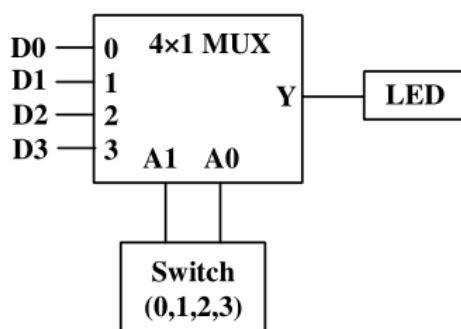
if (A > B) then X = X+Y else X = X-Y

解：



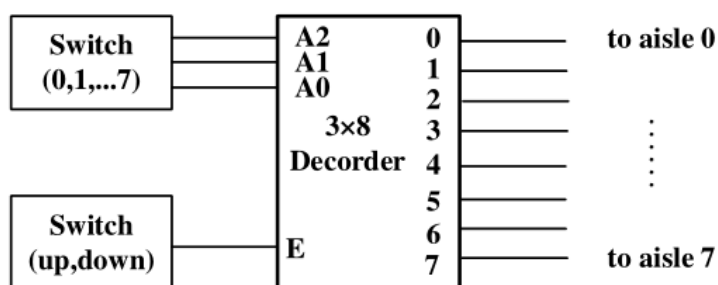
博物馆有 4 个门，每个门都有 1 个传感器，如果门打开，输出 1，门关闭，则输出 0。馆内有 1 个 LED 灯，用来指示门的状态。由于 1 个 LED 只能指示 1 个门的状态，因此管理人员购买了 1 个可以设置为 0、1、2、3 位置的开关，并且具有 2-bit 输出，表示开关的位置。设计检测电路连接 4 个传感器、1 个开关和 1 个 LED 灯，至少使用 1 个数据选择器或译码器，用来指示 4 个门的状态。用功能模块设计该电路，例如“2×1 数据选择器”或“3×8 译码器”，不用设计数据选择器或译码器的内部电路。

解：



超市希望向顾客显示 8 个过道之一的商品打折。所以，在每个过道上方安装 1 盏灯，每盏灯都有 1-bit 的输入，当输入为 1 时，灯点亮。店长有 1 个可以设置为 0、1、2、3、4、5、6、7 位置的开关，并且具有 3-bit 输出，表示开关的位置。第 2 个开关表示超市是否打折，具有 1-bit 输出，此开关打开时输出 1，表示有打折。设计电路连接 2 个开关和 8 盏灯，至少使用 1 个数据选择器或译码器，用来指示 8 个过道的情况。用功能模块设计该电路，例如“2×1 数据选择器”或“3×8 译码器”，不用设计数据选择器或译码器的内部电路。

解：



已知组合电路由以下 3 个逻辑函数定义，用译码器和或门设计实现此电路。

$$F1 = \overline{X+Y} + XY\overline{Z}$$

$$F2 = \overline{X+Y} + \overline{X}YZ$$

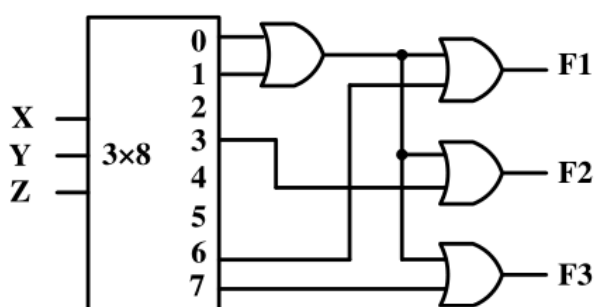
$$F3 = \overline{X+Y} + XYZ$$

解：

真值表

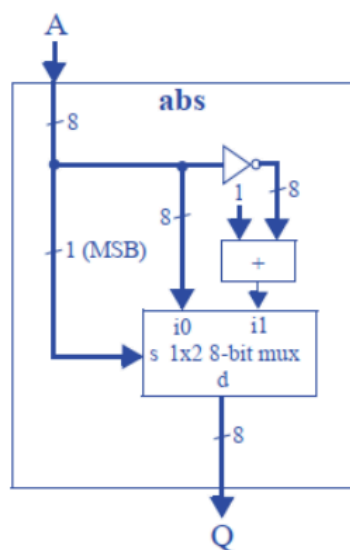
X	Y	Z	F1	F2	F3
0	0	0	1	1	1
0	0	1	1	1	1
0	1	0	0	0	0
0	1	1	0	1	0
1	0	0	0	0	0
1	0	1	0	0	0
1	1	0	1	0	0
1	1	1	0	0	1

逻辑电路



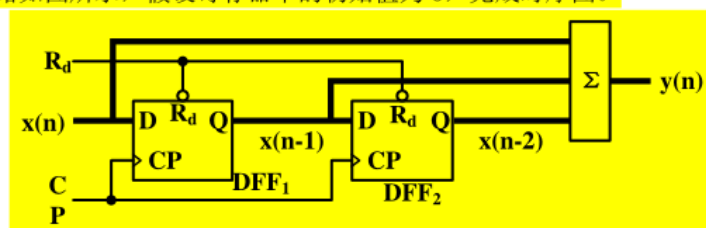
构建一个绝对值运算模块 abs，该模块有 1 个 8-bit 输入 A，A 为有符号二进制数，以及 1 个无符号 8-bit 输出 Q，Q 即为 A 的绝对值。如果输入为 00001111 (+15)，则输出也 00001111 (+15)，但如果输入为 11111111 (-1)，则输出为 00000001 (+1)。

解：



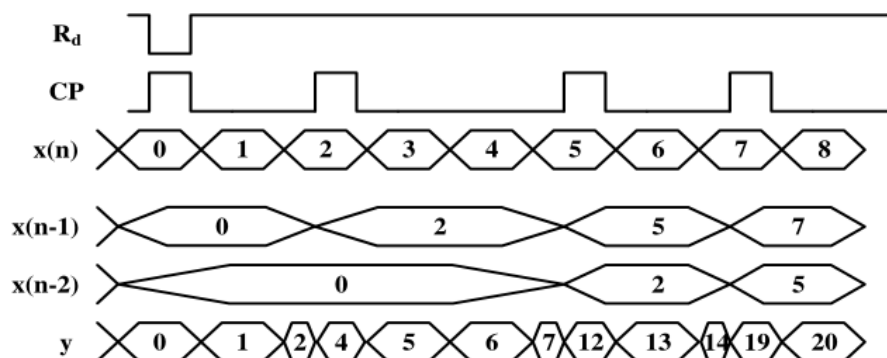
第 4 章 时序逻辑电路

移位寄存器电路如图所示，假设寄存器中的初始值为 0，完成时序图。



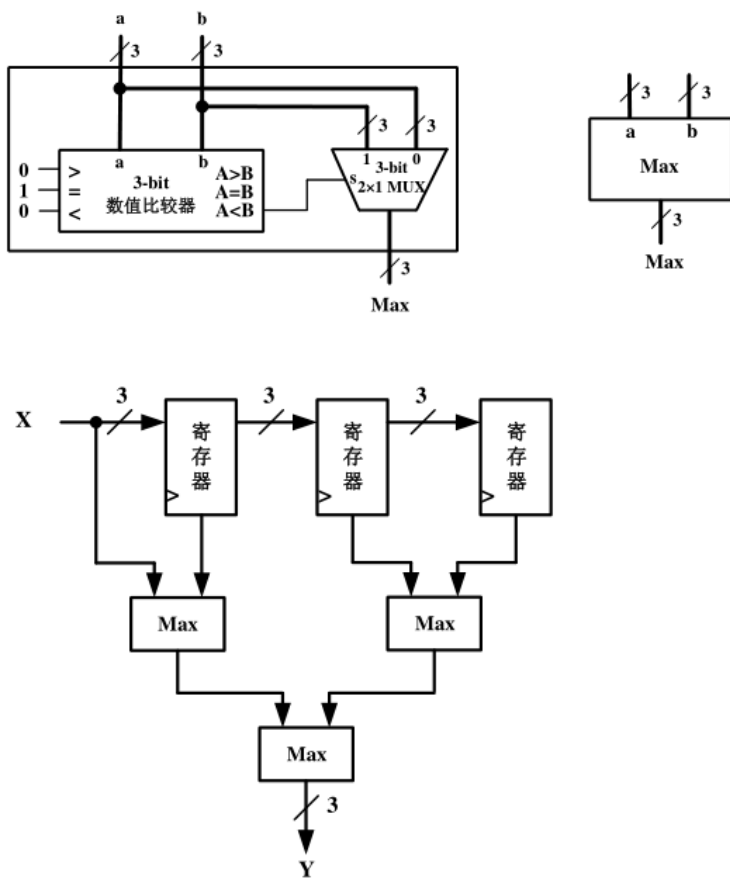
时序图如下：

解：



某温度传感器每 1 秒给出当前测量温度 X （3 位二进制数据），为了减少测量波动的影响，每次将当前值与以前 3 个连续测量值，求最大值作为当前输出值 Y （3 位二进制数据）。设计温度模块信息处理流程。（提示：外部提供周期为 1 秒的时钟，可以使用选择器、比较器、移位寄存器等模块）

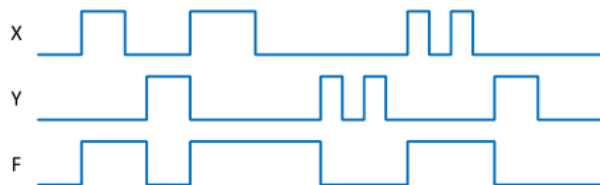
解：



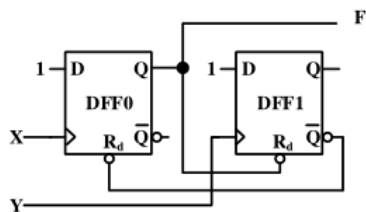
有一个服务窗口，当顾客按动外置按键时，铃声响起。这时，服务人员按动窗口内置按键，铃声停止。请用两个上升沿 D 触发器，两个按键，两个电阻，一个电池和一个响铃实现电路设计。画出电路示意图并简述其工作原理。使用 Verilog HDL 描述电路。

解：

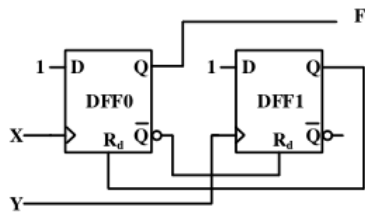
电路为双输入控制脉冲发生器，当 X 的上升沿到达时，F 输出为 1，当 Y 的上升沿到达时，F 输出为 0。即始于 X 上升沿，止于 Y 的上升沿。电阻用于生成按键电路，高电平接电源正极。



如果 Rd 低电平有效，电路如下图：



如果 Rd 高电平有效



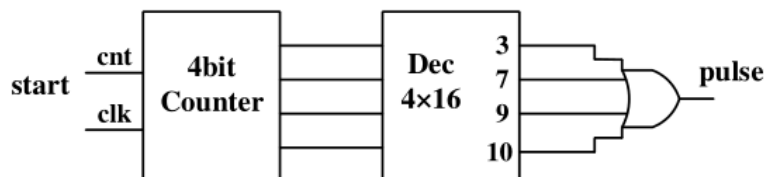
相关代码:

```
module bell_system (
    input wire customer_btn, // 顾客按键
    input wire staff_btn,    // 服务人员按键
    output reg bell           // 响铃输出
);
// 控制逻辑
reg rd0;
always @ (posedge customer_btn or negedge rd0)
    if (!rd0)
        bell <= 0;
    else
        bell <= 1;
always @ (posedge staff_btn or negedge bell)
    if (!bell)
        rd0 <= 1;
    else
        rd0 <= 0;
endmodule
```

电路功能描述如下: 电路工作以 16 个时钟周期循环计数 (分别命名为周期 0, 周期 1, ..., 周期 15), 输出 “PULSE” 信号只在周期 3,7,9,10 输出高电平脉冲。使用计数器, 译码器和逻辑门实现该电路。

解:

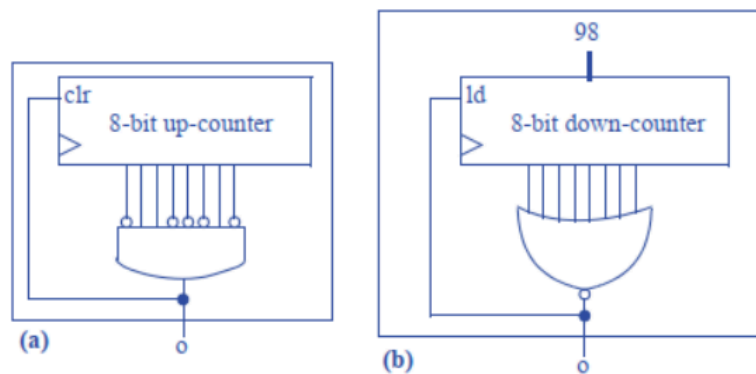
原理:



设计一个每 99 个时钟周期输出 1 的电路:

- 使用有同步清零功能的加法计数器和其他必要的逻辑门实现。
- 使用有并行置数功能的减法计数器和其他必要的逻辑门实现。

解:



某时序电路有一个输入 $w1$ ，一个输出 z ，功能是进行序列检测。如果在连续 4 个周期内， $w1$ 输入 1101，则 z 输出 1，否则 z 输出 0。

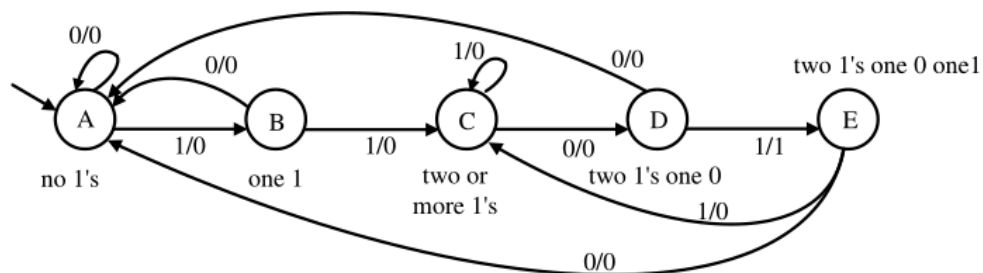
例如：

$w1$	0	1	1	0	1	1	0	1	1	1	1	0	1
z	0	0	0	0	1	0	0	1	0	0	0	0	1

① 该电路是 Mealy 型电路还是 Moore 型电路？

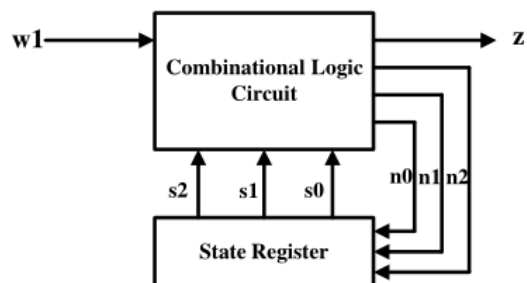
Mealy 型电路

② 试给出完整状态转移图



③ 实现该时序电路

A. 电路实现



共 5 个状态，需要 3bit 编码

A: 000 B: 001 C: 010 D: 011 E: 100

B. Verilog 实现

```
module seq_det(  
    input clk,  
    input reset,  
    input w1,  
    output reg z  
);  
    reg [2:0] current_state;  
    reg [2:0] next_state;  
    //状态声明  
    parameter A = 3'b000,  
    parameter B = 3'b001,  
    parameter C = 3'b010,  
    parameter D = 3'b011,  
    parameter E = 3'b100;  
  
    always @(posedge clk, posedge reset)  
        if(reset)  
            current_state <= A;  
        else  
            current_state <= next_state;  
  
    always @(*)  
        case(current_state)  
            A:  
                begin  
                    z = 0;  
                    if(w1 == 1'b1) next_state = B;  
                    else next_state = A;  
                end  
            B:  
                begin  
                    z = 0;  
                    if(w1 == 1'b1) next_state = C;  
                    else next_state = A;  
                end  
            C:  
                begin  
                    z = 0;  
                    if(w1 == 1'b0) next_state = D;  
                    else next_state = C;  
                end  
            D:  
                begin  
                    z = 0;  
                    if(w1 == 1'b0) next_state = E;  
                    else next_state = C;  
                end  
            E:  
                begin  
                    z = 0;  
                    if(w1 == 1'b1) next_state = A;  
                    else next_state = C;  
                end  
        endcase  
endmodule
```

```

        if(w1 == 1'b1)
        begin
            next_state = E;
            z=1;
        end

        else
        begin
            next_state = A;
            z=0;
        end
    E:
    begin
        z=0;
        if(w1 == 1'b1) next_state = C;
        else          next_state = A;
    end
    default: next_state = A;
endcase
endmodule

```

